

2026

Memoria proyecto app fotografía

CONCURSO FOTOGRAFÍA
ALBERTO NIETO, HÉCTOR JIMENEZ

ÍNDICE

Día 1: 2/06/2026 2

Día 2: 3/06/2026 3

Día 3: 4/06/2026 4

Día 4: 5/06/2026 5

Día 5: 8/06/2026 8

Día 6: 9/06/2026 10

Día 7: 10/06/2026 11

Día 8: 11/06/2026 12

Día 9: 12/06/2026 13

Día 10: 13/06/2026 14

Día 1: 2/06/2026

Hemos creado la acta de inicio del proyecto, en la que incluimos los objetivos, lo que abarcará la aplicación, los roles y las responsabilidades:

- **Objetivos:** diseñar y desarrollar una aplicación funcional que permita gestionar un concurso de fotografía, incluyendo registro de participantes, puntuaciones, seguimiento de resultados y visualización de calificaciones.
- **Alcance:** Base de datos en 3FN, Pantalla para registrar concursos, jueces y participantes, pantalla para subir fotografías y ver resultados, pantalla para poner calificaciones, registro de los Login en un fichero de log, BBDD desplegada en un servidor AWS/Ubuntu con acceso por SSH.
- **Roles:** Alberto: Programación, Interfaz y documentación, Héctor BBDD y servidor.

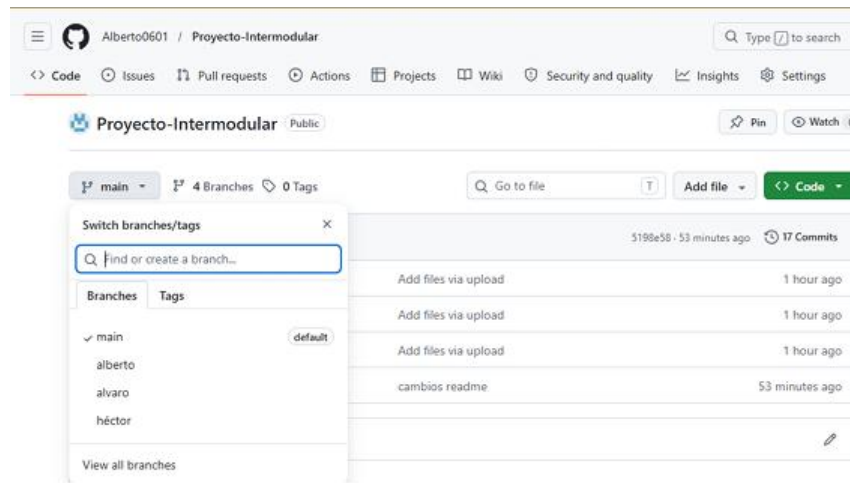
El día uno también hicimos los análisis de riesgos del puesto, los que identificamos que se podían dar son los siguientes:

- **Ergonómicos:**
 - Uso de silla no adecuada o sin soporte lumbar.
 - Postura incorrecta durante largas horas.
 - Falta de reposapiés o apoyo adecuado.
- **Fatiga visual:**
 - Uso prolongado de pantalla sin descansos.
- **Riesgos posturales:**
 - Permanecer sentado durante muchas horas sin moverse.
 - Cruzar piernas o encorvarse.
- **Riesgos eléctricos:**
 - Cables mal organizados.

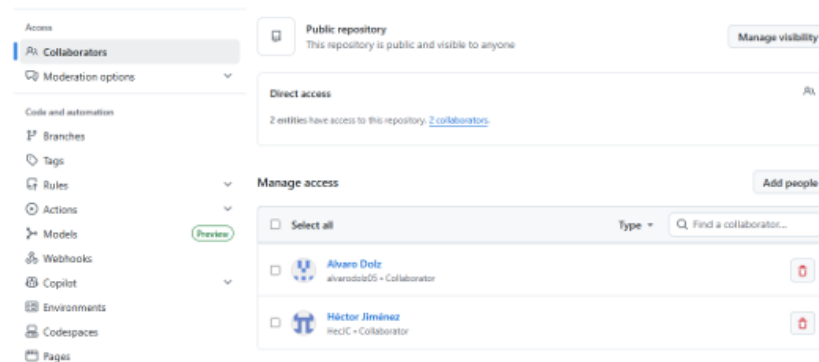
Problemas con el equipo: hemos hecho el trabajo entre dos, por lo que hemos tenido que recortar ideas que teníamos para la aplicación.

Día 2: 3/06/2026

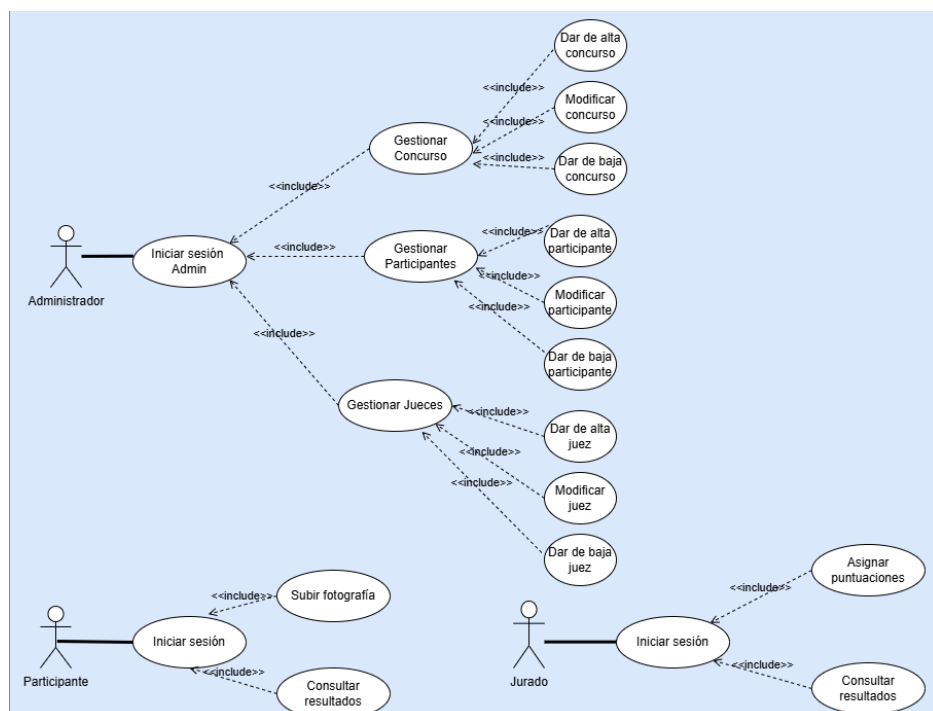
Hemos creado el repositorio de GIT con las ramas de cada miembro:

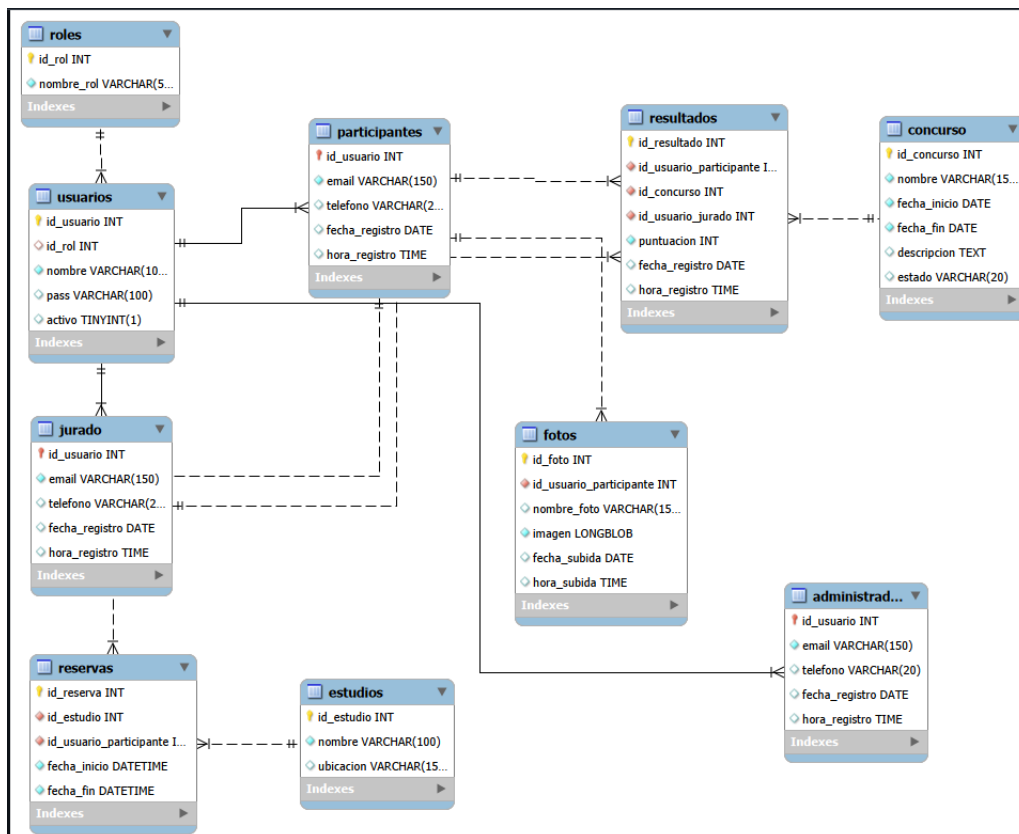


Aquí se muestran los colaboradores:



A continuación hemos hecho los diagramas de Modelo E/R y el de Casos de uso:





Día 3: 4/06/2026

Hoy hemos creado el Script de SQL y este está funcionando, adjunto una parte del script:

```

DROP DATABASE IF EXISTS ConcursoFotografia;
CREATE DATABASE IF NOT EXISTS ConcursoFotografia;
USE ConcursoFotografia;

-- Tabla para normalizar los roles
CREATE TABLE IF NOT EXISTS roles (
    id_rol INT PRIMARY KEY,
    nombre_rol VARCHAR(50) NOT NULL
);

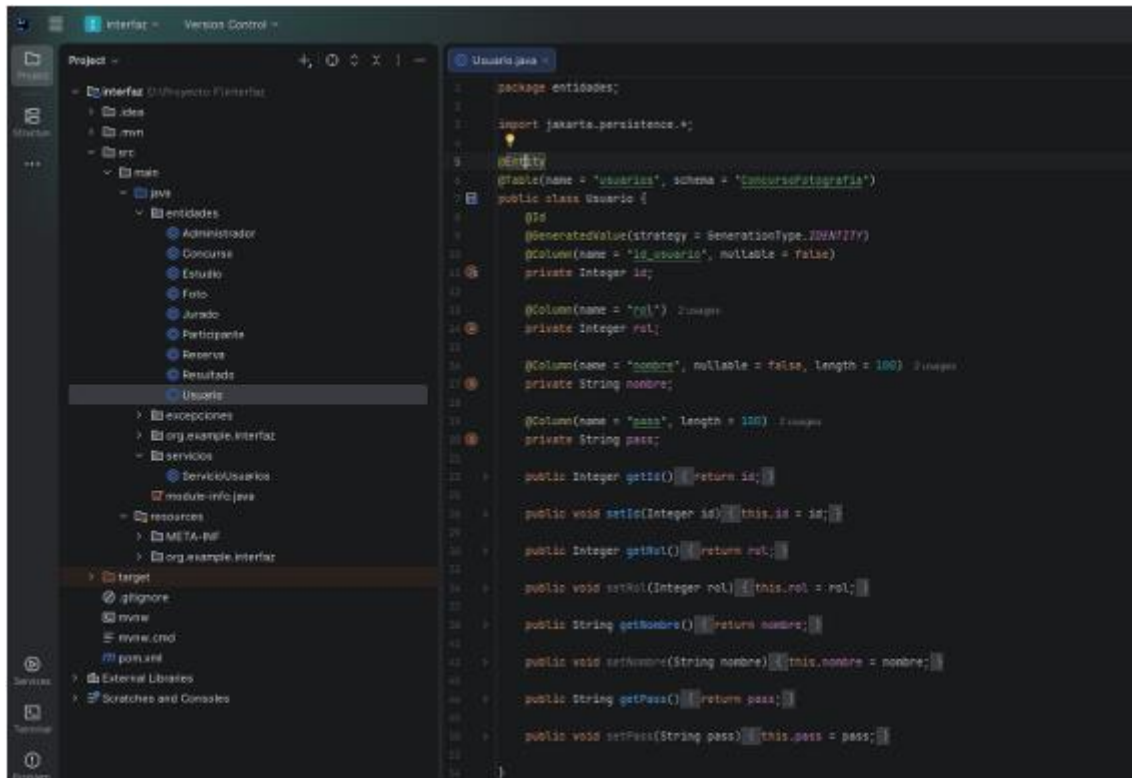
CREATE TABLE IF NOT EXISTS usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    id_rol INT,
    nombre VARCHAR(100) NOT NULL,
    pass VARCHAR(100),
    activo BOOLEAN DEFAULT TRUE, -- Añadí la columna 'activo' para permitir el caso de uso 'Dar de baja'.
    FOREIGN KEY (id_rol) REFERENCES roles(id_rol)
);

CREATE TABLE IF NOT EXISTS administrador (
    id_usuario INT PRIMARY KEY, -- id_usuario es la id de admin, participante y juez
    email VARCHAR(150) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    fecha_registro DATE DEFAULT (CURRENT_DATE),
    hora_registro TIME DEFAULT (CURRENT_TIME),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);

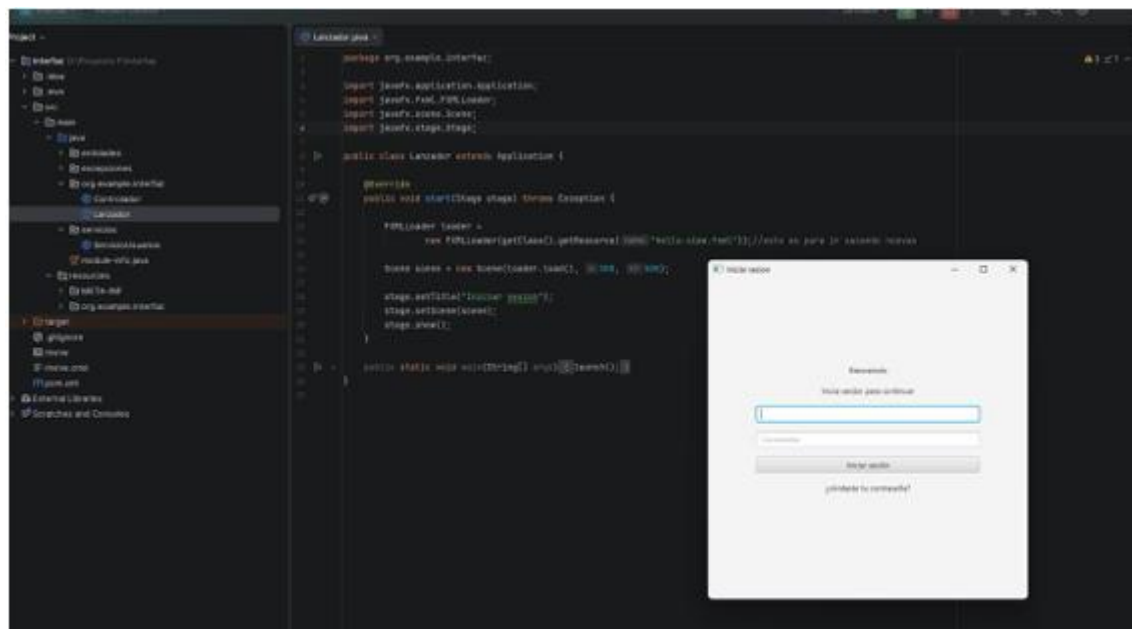
CREATE TABLE IF NOT EXISTS participantes (
    id_usuario INT PRIMARY KEY,
    email VARCHAR(150) UNIQUE NOT NULL,
    telefono VARCHAR(20),
    fecha_registro DATE DEFAULT (CURRENT_DATE),
    hora_registro TIME DEFAULT (CURRENT_TIME),
    FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario)
);
  
```

Día 4: 5/06/2026

Creación del proyecto, le he añadido con JPA las entidades de las tablas:



También he creado la interfaz de Login con java FX y le he añadido funcionalidad con java:



Para que la pantalla de Login tenga funcionalidad he creado una clase de ServicioUsuarios y Controlador:

```
Lanzador.java ServicioUsuarios.java
package servicios;

import javax.persistence.EntityManager;
import javax.persistence.criteria.CriteriaQuery;
import javax.persistence.criteria.Root;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

public class ServicioUsuarios {

    //Constantes atributos
    private final EntityManager em;
    private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(); //encrptador

    public ServicioUsuarios(EntityManager em) {
        this.em = em;
    }

    //metodo

    public Usuario login(String nombre, String passwordPlane) {
        try {
            TypedQuery<Usuario> query = em.createQuery(
                "SELECT u FROM Usuario u WHERE u.nombre = :nombre", Usuario.class
            );
            query.setParameter("nombre", nombre);

            Usuario usuario = query.getSingleResult();

            if (encoder.matches(passwordPlane, usuario.getPass())) {
                return usuario;
            } else {
                System.out.println("Contraseña incorrecta");
                throw new CredencialesInvalidasException();
            }

        } catch (NoResultException e) {
            System.err.println("No existe el usuario en la BD");
            return null;
        } catch (CredencialesInvalidasException e) {
            System.err.println("Error al obtener el inicio de sesión: " + e.getMessage());
            return null;
        }
    }
}
```

Esta clase consulta la tabla usuarios, comprueba las contraseñas hash con BCrypt y te devuelve si la contraseña es correcta o incorrecta, además le he añadido que me lance una excepción personalizada que he creado para cuando hay credenciales invalidas.

```
}else {
    System.out.println("Contraseña incorrecta");
    throw new CredencialesInvalidasException();
}

}catch (NoResultException e){
    System.err.println("No existe el usuario en la BBDD");
    return null;
}catch (CredencialesInvalidasException e){
    System.err.println("Error al obtener el inicio de sesión: " + e.getMessage());
    return null;
}
```

```

Lanzador.java  ServicioUsuarios.java  CredencialesInvalidasException.java x
1 package excepciones;
2
3 public class CredencialesInvalidasException extends Exception { 3 usages
4
5     public CredencialesInvalidasException() { 1 usage
6
7         super("Credenciales no validas, inténtelo de nuevo");
8     }
9
10    public CredencialesInvalidasException(String mensaje) { no usages
11
12        super(mensaje);
13    }
14
15    public CredencialesInvalidasException(String mensaje, Throwable causa) { no usages
16
17        super(mensaje, causa);
18    }
19 }

```

Aquí la clase Controlador que es la que conecta la funcionalidad de JavaFX con Java:

```

Lanzador.java  Controlador.java
12 public class Controlador {
13     private TextField txtUsuario;
14
15     @FXML
16     private PasswordField txtPassword;
17
18     @FXML
19     private Label lblMensaje;
20
21     @FXML
22     protected void iniciarSesion() { //método para iniciar sesión
23         //semitizar la string de usuario y password, importante decir que es de login
24         //si existe el usuario pasar a la siguiente ventana y llevarte al siguiente controlador los objetos que necesite
25         String usuario = txtUsuario.getText();
26         String password = txtPassword.getText();
27
28         try (EntityManager em = Persistence.createEntityManagerFactory("concursofotos").createEntityManager();) {
29             ServicioUsuarios u = new ServicioUsuarios(em);
30             Usuario log = u.login(usuario, password);
31             if (log != null) { //si el usuario existe
32                 System.out.println(log.getNombre());
33                 if (log.getRol() == 1) {
34                     System.out.println("Rol 1 administrador");
35                     //que te mande a la vista de admin-->su pantalla
36                     //controlador siguiente vista, mandar con un método el objeto y luego otro método para mostrar
37                     if (log.getRol() == 1) {
38                         System.out.println("Rol 2 jueces");
39                         //que te mande a la vista de juez-->su pantalla
40                     } else if (log.getRol() == 3) {
41                         System.out.println("Rol 3 participantes");
42                         //que te mande a la vista de participante-->su pantalla
43                     }
44                 } else {
45                     System.out.println("usuario no encontrado");
46                 }
47             }
48         } catch (Exception e) {
49             System.err.println(e.getMessage());
50         }
51     }
52 }

```


Día 5: 8/06/2026

He creado la interfaz de Login, además de las interfaces de cada usuario. Aquí se ve el método para cambiar de ventana:

```
private void cambiarDeVentana(String fxml, Usuario usuario) throws IOException { 3 usages Alberto +1

    FXMLLoader fxmload = new FXMLLoader(getClass().getResource(fxml)); //objeto clase FXMLLoader

    Scene scene = new Scene(fxmload.load()); //objeto de la clase Scene

    Object controladorDest = fxmload.getController(); //me da el controlador de la ventana que acaba
    if (controladorDest instanceof ControladorAdministrador) { //esto de aqui es para llevarme el ob
        ((ControladorAdministrador) controladorDest).setUsuarioAdmin(usuario);
    } else if (controladorDest instanceof ControladorJuez) {
        ((ControladorJuez) controladorDest).setUsuarioJuez(usuario);
    } else if (controladorDest instanceof ControladorParticipante) {
        ((ControladorParticipante) controladorDest).setUsuarioParticipante(usuario);
    }

    Stage stg = (Stage) txtUsuario.getScene().getWindow();
    stg.setScene(scene);
    stg.show(); //mostrar
}
```

Aquí el de iniciar sesión:

```
@FXML Alberto +1
protected void iniciarSesion() { //metodo para iniciar sesion
    //sanitizar la string de usuario y password, importante decir que es de login
    //si existe el usuario pasar a la siguiente ventana y llevarte al siguiente controlador los objetos que necesi
    String usuario = txtUsuario.getText();
    String password = txtPassword.getText();

    try (EntityManager em = Persistence.createEntityManagerFactory("concursoFotos").createEntityManager()) {

        ServicioUsuarios u = new ServicioUsuarios(em);
        Usuario log = u.login(usuario, password);

        if (log != null) { //si el usuario existe
            System.out.println(log.getNombre());
            if (log.getIdRol().getId() == 1) {
                System.out.println("Rol 1 administrador");
                cambiarDeVentana("FXML: /org/example/interfaz/administrador-view.fxml", log); //la ruta donde está el
                //que te mande a la vista de admin--> su pantalla
                //controlador siguiente vista, mandar con un metodo el objeto y luego otro metodo para mostrar
            } else if (log.getIdRol().getId() == 2) {
                System.out.println("Rol 2 juezas");
                //que te mande a la vista de juez--> su pantalla
                cambiarDeVentana("FXML: /org/example/interfaz/juez-view.fxml", log);
            } else if (log.getIdRol().getId() == 3) {
                System.out.println("Rol 3 participante");
            }
        }
    }
}
```

En el FXML tengo el estilo:

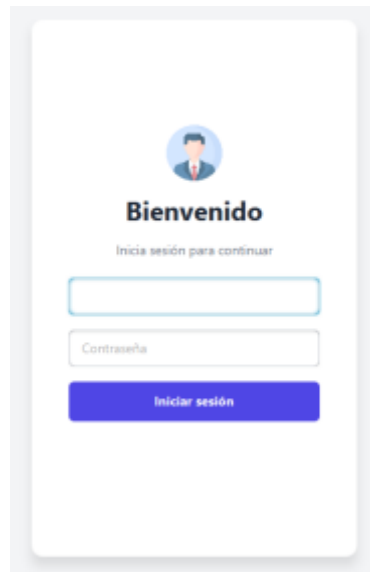
```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.image.*?>
<?import javafx.scene.layout.*?>

<StackPane xmlns:fx="http://javafx.com/fxml"
    fx:controller="org.example.interfaz.Controlador"
    style="-fx-background-color: #f3f4f6;
    -fx-font-family: 'Segoe UI', Arial, sans-serif;">

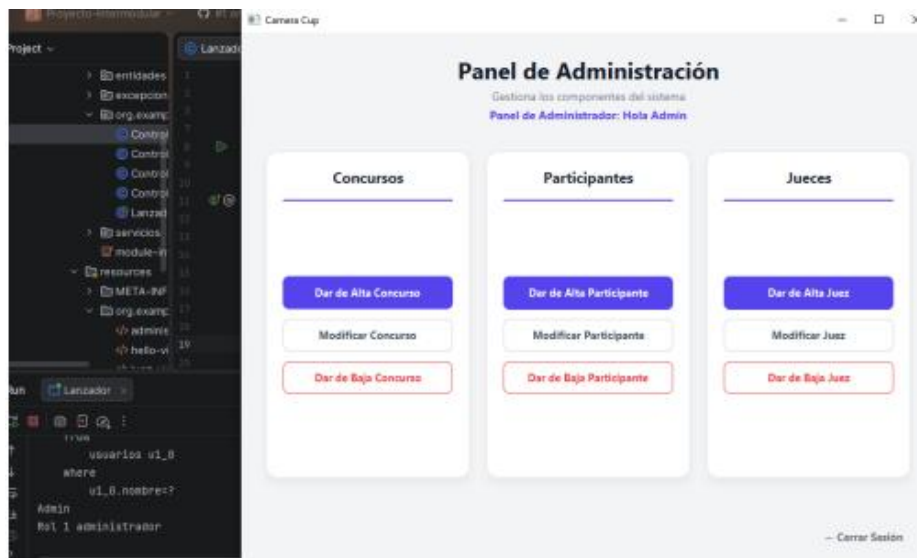
    <children>
        <VBox alignment="CENTER"
            maxWidth="350"
            maxHeight="300"
            spacing="10"
            style="-fx-background-color: #ffffff;
            -fx-padding: 30px 40px;
            -fx-background-radius: 10px;
            -fx-effect: dropshadow(three-pass-box, rgba(0, 0, 0, 0.1), 15, 0, 0, 10);">

            <children>
                <ImageView fitWidth="60"
                    fitHeight="60"
                    preserveRatio="true">
```

Resultado:



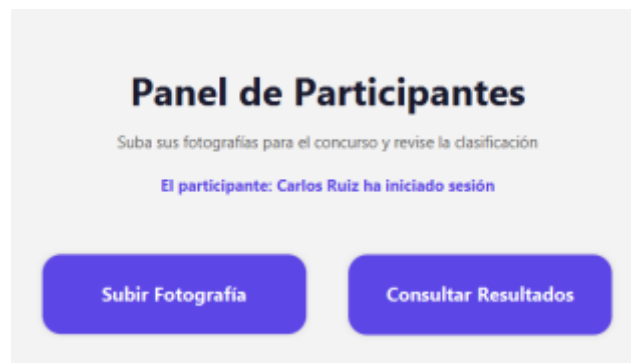
Aquí se ve que conecta correctamente con la BBDD, además las interfaces de usuarios muestran que usuarios están conectados. También hay un botón de cerrar sesión que te devuelve al Login.



Pantalla de jueces:



Pantalla de participantes:



Día 6: 9/06/2026

Creación del fichero log y el servicio que lo gestiona:

```
package servicios;

import java.io.*;
import java.time.*;

public class ServicioLog {

    private static final String ARCHIVO_DE_LOG = "logs.log";
    private static final DateFormatter formateador = DateFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public static void registrar(String nivel, String mensaje) {
        String fechaHora = LocalDateTime.now().format(formateador);
        String linealog = String.format("%s [%s] %s", fechaHora, nivel, mensaje);

        try (BufferedWriter bw = new BufferedWriter(new FileWriter(ARCHIVO_DE_LOG, true))) {
            bw.write(linealog);
            bw.newLine();
        } catch (IOException e) {
            System.err.println("Error al escribir en el archivo de log: " + e.getMessage());
        }
    }
}
```

Aquí están las partes donde he añadido el método de ServicioLog:

```
try (EntityManager em = Persistence.createEntityManagerFactory("concursoFotos").createEntityManager();) {

    ServicioLog.registrar(nivel: "INFORMACIÓN", mensaje: "Intento de inicio de sesión: " + usuario);
    ServicioUsuarios u = new ServicioUsuarios(em);
    Usuario log = u.login(usuario, password);

    if (log != null) { //si el usuario existe
        ServicioLog.registrar(nivel: "INFO", mensaje: "Login EXITOSO. Usuario: " + log.getNombre() + " - Rol ID: " + log.getIdRol().getId());
        System.out.println(log.getNombre());
    }
    else {
        ServicioLog.registrar(nivel: "ADVERTENCIA", mensaje: "Login FALLIDO. Credenciales incorrectas para: " + usuario);
        System.out.println("Usuario no encontrado");
    }
} catch (Exception e) {
    ServicioLog.registrar(nivel: "ERROR", mensaje: "Error critico en la base de datos al logear a " + usuario + ": " + e.getMessage());
    System.err.println(e.getMessage());
}
```

La primera vez es para saber quién intenta iniciar sesión, la segunda vez para recoger la información del usuario, la tercera vez es cuando falla el inicio de sesión y la última vez cuando hay un error.

Aquí se ve el archivo .log que tiene recogidos todos los inicios de sesión.

```
[2026-06-09 11:00:10] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 11:00:10] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 11:01:14] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 11:01:14] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 11:11:18] [INFORMACIÓN: ] Intento de inicio de sesión: Carlos Ruiz
[2026-06-09 11:11:18] [INFO] Login EXITOSO. Usuario: Carlos Ruiz - Rol ID: 3
[2026-06-09 12:03:50] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 12:03:51] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 12:43:12] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 12:43:12] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 12:48:41] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 12:48:41] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 19:23:15] [ERROR] Error crítico en la base de datos al loguear a admin: Unable to create requested service [org.hibernate.engine.jdbc.env.spi.JdbcEnvironment]

The last packet sent successfully to the server was 0 milliseconds ago. The driver has not received any packets from the server.] [Communications link failure]

The last packet sent successfully to the server was 0 milliseconds ago. The driver has not received any packets from the server.] [n/a]
[2026-06-09 19:24:24] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 19:24:24] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 19:44:14] [INFORMACIÓN: ] Intento de inicio de sesión: admin
[2026-06-09 19:44:15] [INFO] Login EXITOSO. Usuario: Admin - Rol ID: 1
[2026-06-09 19:59:27] [INFORMACIÓN: ] Intento de inicio de sesión: Ana López
[2026-06-09 19:59:28] [INFO] Login EXITOSO. Usuario: Ana López - Rol ID: 2
```

Día 7: 10/06/2026

Creación del Javadoc y la Landing Page:

MODULE TREE INDEX SEARCH HELP

org.example.interfaz

Search documentation (type /)

Contents

Description

Packages

Module org.example.interfaz

module org.example.interfaz

Packages

All Packages Exports Opens

Package	Exported To Modules	Opened To Modules	Description
entidades	None	All Modules	
org.example.interfaz	All Modules	None	

Contents

Description

Classes and Interfaces

Package org.example.interfaz

package org.example.interfaz

Classes

Class	Description
AlertaHelper	
Controlador	
ControladorAdministrador	
ControladorAsignarPuntuacion	
ControladorBajaConcurso	
ControladorBajaJuez	
ControladorBajaParticipante	
ControladorFormularioConcurso	
ControladorFormularioUsuario	
ControladorJuez	
ControladorModificarConcurso	
ControladorModificarJuez	
ControladorModificarParticipante	
ControladorParticipante	
ControladorSubirFoto	
ControladorVerResultados	
ControladorVerResultadosJuez	
Lanzador	

Sistema de Gestión de Concursos Fotográficos

Plataforma web diseñada para gestionar concursos fotográficos mediante un sistema de roles que permite administrar concursos, participar mediante fotografías y evaluar imágenes de forma organizada y transparente.

[Ver funcionamiento](#)

¿Qué es la aplicación?

Esta aplicación permite gestionar de forma integral concursos de fotografía desde una única plataforma.

El sistema facilita la administración de concursos, la participación mediante la subida de imágenes y la evaluación por parte de jueces autorizados.

Objetivos principales

- Centralizar la gestión de concursos.
- Facilitar la participación de usuarios.
- Permitir evaluaciones objetivas.
- Automatizar clasificaciones.
- Garantizar transparencia en resultados.

Roles del Sistema

Cada usuario dispone de permisos específicos según su función.

Administrador

- Gestión de concursos (dar de alta/baja y actualizarlos).
- Gestionar usuarios (dar de alta/baja y

Participante

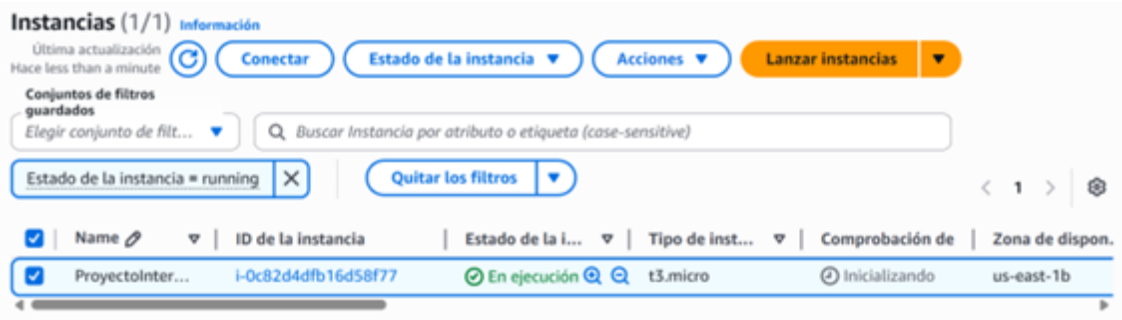
- Subir fotografías.
- Consultar puntuaciones.

Juez

- Poner calificaciones.
- Consultar puntuaciones.

Día 8: 11/06/2026

Se terminó el servidor para subir la BBDD y la Landing page:



Amazon) Información

Una AMI posee el sistema operativo, el servidor de aplicaciones y las aplicaciones de la instancia. Si a continuación no ve una AMI adecuada, utilice el campo de búsqueda o elija **Buscar más AMI**.

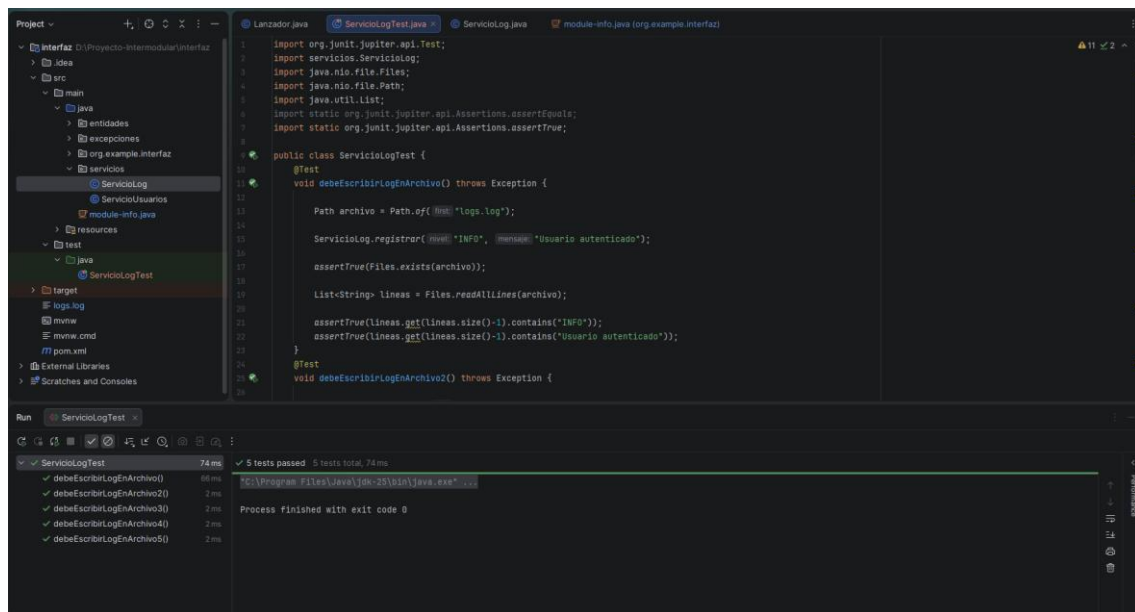
Busque en nuestro catálogo completo que incluye miles de imágenes de sistemas operativos y aplicaciones

Recientes

Inicio rápido



También se ha creado el .jar de la aplicación y funciona.



Día 10: 13/06/2026

Hemos entregado todo: Repositorio GIT en Zip + enlace, Script SQL, Diagramas, Javadoc, Riesgos del puesto (incluido en el día 1 de este documento), Capturas del despliegue, la memoria y la presentación.